

LABORATOR 9

FLUXURI DE DATE ÎN JAVA

1.1. Noțiuni teoretice

Pentru a stoca datele după terminarea programului e nevoie ca ele să fie ținute în fișiere, baze de date, etc.

Java oferă clasa **File** pentru operații cu fișiere dar nu pentru a citi sau scrie conținutul fișierelor. Obiectele de tip **File** pot descrie fișiere sau directoare.

Constructorii pentru clasa **File**:

- ☐ `File(String cale)`
- ☐ `File(File director, String fisier)`
- ☐ `File(String director, String fisier)`

Fluxuri de date

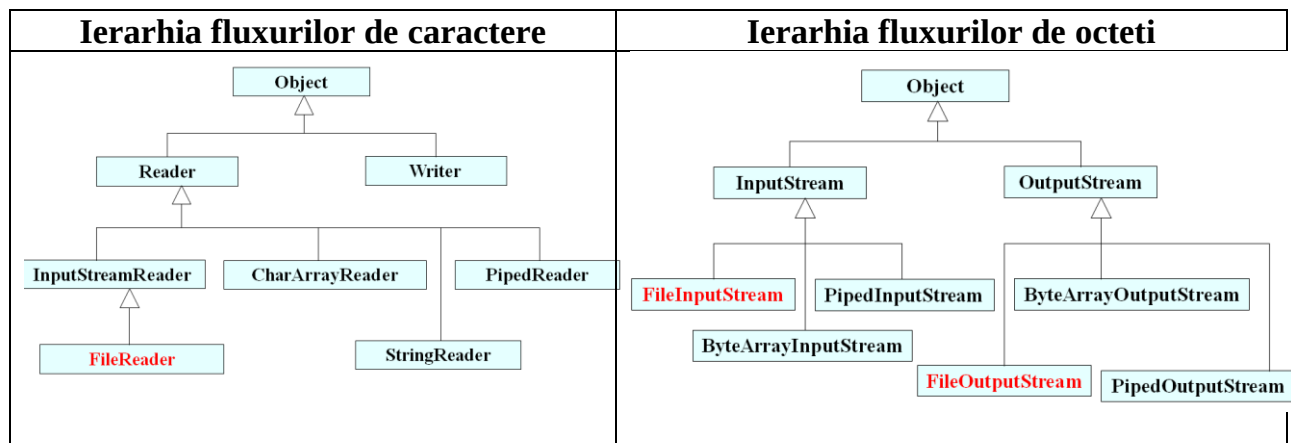
Toate clasele pentru lucrul cu fișiere se găsesc în pachetul **java.io**. Există două abordări în lucrul cu fișiere:

- ☐ Accesul aleator la fișiere
- ☐ Tratarea fișierelor ca un flux de date

Într-un flux (canal de comunicație) datele pot curge într-o singură direcție. Orice flux de date are o sursă (care poate doar scrie date în flux) și o destinație (care poate doar citi date din flux)

Fluxurile de date pot fi împartite în două categorii, după poziția programului față de flux:

- ☐ **Fluxuri de intrare (pentru citirea datelor)** – programul este destinația fluxului iar sursa este un fișier, un array, un alt program
- ☐ **Fluxuri de ieșire (pentru scrierea datelor)** – programul este sursa fluxului iar destinația este un fișier, un array sau un alt program



Buffering – este o tehnica ce creste performanta lucrului cu fisiere prin scrierea/citirea unui bloc de date si nu doar a unui octet.

Fluxurile pentru citirea/scrierea efectiva a datelor într-un fișier sunt:

Fluxuri caractere	Fluxuri octeti
FileReader, FileWriter	FileInputStream, FileOutputStream

Clasele ce descriu fluxuri pentru procesarea datelor pot fi împartite în functie de tipul de procesare pe care îl efectueaza:

Fluxuri caractere	Fluxuri octeti
BufferedReader, BufferedWriter	BufferedInputStream, BufferedOutputStream

Fluxurile pentru procesarea datelor sunt folosite pentru a introduce un buffer în procesul de scriere/citire a informatiilor, reducând astfel numarul de accese la dispozitivul ce reprezinta sursa originala de date. Sunt mult mai eficiente decât fluxurile fara buffer si din acest motiv se recomanda folosirea lor ori de câte ori este posibil.

Serializarea obiectelor

Serializarea este posibilitatea de a citi si scrie obiecte din memorie in fluxuri de date. Serializarea este transformarea unui obiect într-o secvență de octeți, din care să poată fi refăcut ulterior obiectul original.

Pentru a putea serializa un obiect, clasa din care face parte trebuie sa implementeze interfata **Serializable**.

Clasele folosite pentru serializarea obiectelor sunt:

- ☐ ObjectOutputStream
- ☐ ObjectInputStream

Metode folosite pentru serializarea obiectelor sunt:

- ☐ writeObject, pentru scriere (serializare) și
- ☐ readObject, pentru restaurare (deserializare).

Pentru a putea fi serializat un obiect trebuie sa fie instanta a unei clase care implementeaza una din interfețele :

- ☐ java.io.Serializable sau
- ☐ java.io.Externalizable (care extinde clasa Serializable)

1.2. Aplicații propuse

1. Să se implementeze o aplicație Java ce permite crearea interfeței din fig.1. Aplicația citește dintr-un fișier o listă ce cuprinde mai multe numere întregi (scrise pe linii separate) și afișează numărul de numere impare (intr-o componenta de tip TextField). De asemenea, afișați numerele impare într-un fișier, dar si in componenta de tip TextArea.

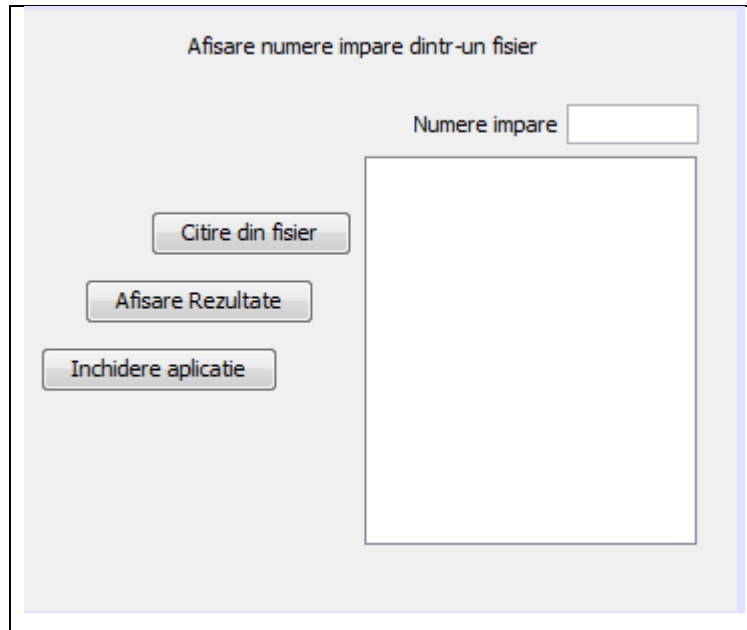


Fig. 1. Interfața grafică utilizator pentru afisarea numerelor impare.

2. Să se implementeze o aplicație Java ce permite crearea interfeței din fig.2. Aplicația va permite efectuarea operațiilor aritmetice principale. Operațiile aritmetice efectuate se vor scrie într-un fisier.



Fig. 2. Interfața grafică utilizator pentru realizarea unor operatii aritmetice principale.

3. Sa se defineasca clasa *Persoana* ce are urmatoarele date membre: *nume*, *varsta*, *salar*. Sa se defineasca minim doi constructori, functii de acces la datele membre de tip get si de tip set si o functie pentru afisarea datelor.

Interfata grafica va contine cel putin 2 butoane (validare si afisare in liste).

La accesarea butonului de validare se vor prelua informatiile introduse in componentele de tip `TextField`, iar cand se va apasa butonul de afisare se vor afisa informatiile despre persoanele introduse intr-o componenta de tip `TextArea`. La preluarea datelor se vor face verificari astfel inca in fiecare componenta de tip `TextField` sa se introduca informatii corecte pentru *nume*, *varsta* si *salar*. De asemenea daca persoana respectiva a mai fost introdusa se va verifica si se va afisa un mesaj corespunzator, persoana nemaifiind adaugata in componenta de tip `TextArea`.